

몰입형 XR을 위한 상황 인식 기반 Adaptive Passthrough Framework

Context-aware Adaptive Passthrough Framework for Immersive XR



팀명: Team VR

202355532 따다소 thadarsoe730@pusan.ac.kr

202355667 최아영 ayoun0421@pusan.ac.kr

202155583 이승주 maya0108@pusan.ac.kr

지도교수: 이명호 교수님

제출일: 2026년 04월 27일

부산대학교

목차

1	서론	1
1.1	프로젝트 소개	1
1.2	프로젝트 배경 및 목표	1
2	관련 연구 및 기존 기술 분석	3
2.1	기존 VR 안전 시스템 및 Passthrough 기술	3
2.2	기존 기술의 제약 및 문제점	3
2.3	본 프로젝트의 차별성	4
3	시스템 개요	5
3.1	전체 시스템 개념 및 흐름도	5
3.2	사용 기술 및 전체 레이어 정의	6
3.2.1	VR Client Layer	6
3.2.2	AI/ML Layer	6
3.2.3	Backend Layer	7
3.2.4	Data Layer	7
3.2.5	Dashboard Layer	7
3.3	상황 인식 기반 위험 분석 및 시각화	7
3.3.1	위험도 산출을 위한 기본 입력 Feature 정의	8
3.3.2	충돌 위험도($R_{collision}$)	9
3.3.3	정적 경계 기반 위험도 (R_{static})	11
3.3.4	AI 기반 동적 객체 충돌 및 접근 의도 예측 기반 위험도 ($R_{dynamic}, R_{intent}$)	12
3.3.5	다양한 Passthrough 시각화 방식	14
3.4	ML 기반 개인화 Passthrough 제어	15
3.4.1	로그 수집 구조	15
3.4.2	개인화 모델 구조	16

3.4.3	학습 레이블 생성	16
3.4.4	온디바이스 배포 및 반영	17
4	비교 평가 계획	18
5	기타	19
5.1	개발 일정 및 역할 분배	19
5.2	기대 효과	20

제1장 서론

1.1 프로젝트 소개

본 프로젝트는 Meta Quest 기반의 몰입형 Extended Reality (이하 XR) 환경에서 사용자의 안전과 몰입감을 동시에 확보하기 위한 **상황 인식 기반 Adaptive Passthrough Framework 프로토타입**을 개발하는 것을 목표로 한다.

본 시스템은 Head Mounted Display (이하 HMD)의 카메라 및 센서 데이터를 수집하여 실시간으로 분석하는 시스템을 통해 사용자의 이동 속도 및 시선, 행동 패턴 등 위험 상황을 자동으로 감지하고 이에 따라 Passthrough를 동적으로 제어한다. 또한 본 시스템은 단순 정적 경계 기반 제어를 넘어, 주변 사람, 이동 물체와 같은 동적 객체의 움직임과 사용자와의 상대적 관계를 함께 분석하여 잠재적인 충돌 가능성과 접근 의도를 예측하는 것을 목표로 한다. 특히 모든 현실 객체를 항상 노출하지 않고, 위험 상황에서만 최소한의 현실 인식을 제공하는 상황 인식(Context-aware) 기반 Adaptive Passthrough 구조를 제안한다.

이를 위해 사용자 행동 로그와 Passthrough 작동 이력을 지속적으로 수집하고, 이를 머신러닝 (이하 ML) 기반 개인화 모델에 반영하여 사용자마다 다른 공간 환경과 행동 패턴에 점진적으로 적응하는 몰입형 XR 안전 시스템을 구현하는 것을 핵심 목표로 삼는다.

1.2 프로젝트 배경 및 목표

XR 시장은 2021년 약 190억 달러 규모에서 2026년 약 1,008억 달러로 연평균 39.7%의 성장이 예측되며, 게임·소셜 등 엔터테인먼트부터 교육·산업·의료까지 활용 범위가 빠르게 넓어지고 있다[1]. 그러나 국내 XR 산업은 콘텐츠 제작·공급 중심에 편중되어 있어 디바이스 및 핵심 기술 분야의 경쟁력 강화가 과제로 지적된다[1].

이러한 시장 성장과 함께 Meta Quest를 비롯한 주요 VR HMD의 보급도 빠르게 확산되고 있으나 기기 사용 환경에서의 안전 문제는 여전히 과제로 남아 있다. 현재 주요 VR HMD에서는 사용자가 미리 설정한 경계선이나 물리적 장애물에 근접하면 Passthrough 모드를 활성화하여 안전을 확보한다[2]. 그러나 방이 협소하거나 벽에 인접하여 활동할 경우, 사용자의

의도와 상관없이 Passthrough가 너무 빈번하게 활성화되어 가상 환경에서의 몰입감이 저해되는 문제가 있다. 특히 게임 콘텐츠의 경우 정적·동적 상황이 실시간으로 교차하는데, 단순 On/Off 방식은 불필요한 간섭을 초래한다.

한편 2025년 말 공개된 Passthrough Camera API는 Quest 3 및 Quest 3S의 전방 카메라를 활용한 컴퓨터 비전 및 ML 파이프라인 구성을 가능하게 했으나[3], 상황 인식 기반의 자동 제어에 활용한 연구는 아직 초기 단계이다.

이에 본 프로젝트는 정적 경계 위험뿐 아니라 주변 사람 및 이동 객체와 같은 동적 위험 요소까지 통합적으로 분석하여, 사용자와의 충돌 가능성과 상호작용 의도를 예측하는 상황 인식 기반 Adaptive Passthrough Framework를 제안한다.

또한 위험 수준과 상황 맥락에 따라 부분 Passthrough, 반투명 객체 표현, Augmented Virtuality 기반 시각화 등 다양한 현실 인식 방식을 선택적으로 제공함으로써, 안전성과 몰입감 사이의 균형을 유지하는 XR 경험 제공을 목표로 한다.

제2장 관련 연구 및 기존 기술 분석

2.1 기존 VR 안전 시스템 및 Passthrough 기술

현재 Meta Quest 시리즈를 비롯한 주요 VR HMD에서는 사용자 안전을 위해 Guardian 시스템을 제공하고 있다. Guardian은 사용자가 물리적 공간에 가상 경계를 직접 설정하고, 경계에 근접할 때 시각 및 햅틱 알림을 제공하는 방식으로 동작하며, Stationary 모드와 Roomscale 모드를 지원한다. Meta Quest 3에서는 이를 발전시킨 Smart Guardian이 도입되어, 카메라 어레이로 방을 자동 스캔하고 경계 설정을 보조하는 보다 직관적인 경험을 제공한다.

Passthrough 기술 측면에서는 카메라를 통해 외부 환경을 실시간으로 헤드셋 화면에 표시하는 고품질 컬러 Passthrough가 Quest 3에서 본격 지원되며 또한 최근 공개된 Passthrough Camera API를 통해 Quest 3 및 Quest 3S의 전방 카메라에 직접 접근하여 컴퓨터 비전 및 ML 파이프라인을 구성할 수 있는 환경이 마련되었다[3].

2.2 기존 기술의 제약 및 문제점

현재의 Guardian 기반 Passthrough 시스템 및 관련 기술은 다음과 같은 한계를 지닌다.

첫째, **Passthrough 활성화 방식이 단순 On/Off 이진 제어에 머물러 있다.** Guardian은 사전에 설정된 경계에 근접할 경우 Passthrough를 활성화하는 방식으로, 공간이 협소하거나 벽에 인접하여 활동하는 환경에서는 사용자의 의도와 무관하게 Passthrough가 지나치게 빈번하게 활성화되어 가상 환경에서의 몰입감이 저해된다. 특히 게임 콘텐츠처럼 정적·동적 상황이 실시간으로 교차하는 경우, 단순한 On/Off 방식은 불필요한 간섭을 초래한다.

둘째, **경계 제어가 수동 설정 또는 개발자 의존적 구조에 머물러 있다.** 일반 사용자가 기기 설정에서 Guardian을 완전히 비활성화하려면 개발자 모드를 별도로 활성화해야 하며, 이는 일반 사용자에게 적합하지 않다[4]. Meta XR Core SDK v66 이후 MR 앱에 한해 개발자가 앱 단에서 경계를 비활성화할 수 있게 되었으나[5], 이는 어디까지나 개발자가 앱을 별도로 구성해야 가능한 방식으로, 사용자가 상황에 따라 능동적으로 제어하는 구조는 아니다.

셋째, **상황 인식 기반의 능동적 대응이 부재하다.** 현재 시스템은 사용자의 이동 속도, 행동

패턴 등을 실시간으로 분석하여 위험도를 판단하는 기능을 제공하지 않으며, 사용자 개인의 행동 이력을 반영한 개인화된 제어도 이루어지지 않는다. 건축 협업 도구인 Arkio의 경우 이동 속도 임계값(0.2m/s)을 기준으로 Passthrough를 자동 전환하는 규칙을 적용하고 있으나[6], 이는 단일 고정 임계값에 기반한 단순 규칙으로 사용자나 상황에 따른 적응적 판단은 이루어지지 않는다.

2.3 본 프로젝트의 차별성

본 프로젝트는 다음과 같은 차별화된 접근 방식을 제안한다.

첫째, **규칙 기반과 ML을 결합한 상황 인식 자동 Passthrough 제어**를 도입한다. HMD에서 수집한 사용자 경계 정보와 센서 데이터를 바탕으로 주변 장애물, 사용자 이동 속도 및 행동 패턴을 실시간 분석하여 위험도를 판단한다. 단순 On/Off 이진 제어나 Arkio와 같은 고정 임계값 방식과 달리, 규칙 기반 로직을 기본 안전망으로 삼고 그 위에 ML 모델을 결합하여 상황에 따른 최적의 Passthrough 제어를 수행한다. 이를 통해 불필요한 Passthrough 활성화를 줄이고 몰입감과 안전을 동시에 확보한다.

둘째, **ML 기반 사용자 개인화 제어**를 구현한다. Passthrough 작동 이력 및 사용자의 암묵적 피드백(수동 개입 여부 등)을 로그로 지속 축적하고, 이를 ML 모델의 학습 데이터로 활용하여 사용자마다 다른 환경과 행동 패턴에 점진적으로 최적화되는 개인 맞춤형 제어 모델로 발전시킨다. 기존 시스템이 모든 사용자에게 동일한 경계 기준을 적용하는 것과 대조적으로, 본 시스템은 사용 이력이 쌓일수록 개인에게 최적화된 제어를 제공한다.

이러한 접근 방식을 통해 기존의 수동·이진 제어 패러다임을 탈피하여 상황 인식 기반의 안정성과 ML 기반의 적응성을 결합한 능동적이고 지능적인 XR 안전 경험을 제공하는 것을 목표로 한다.

제3장 시스템 개요

3.1 전체 시스템 개념 및 흐름도

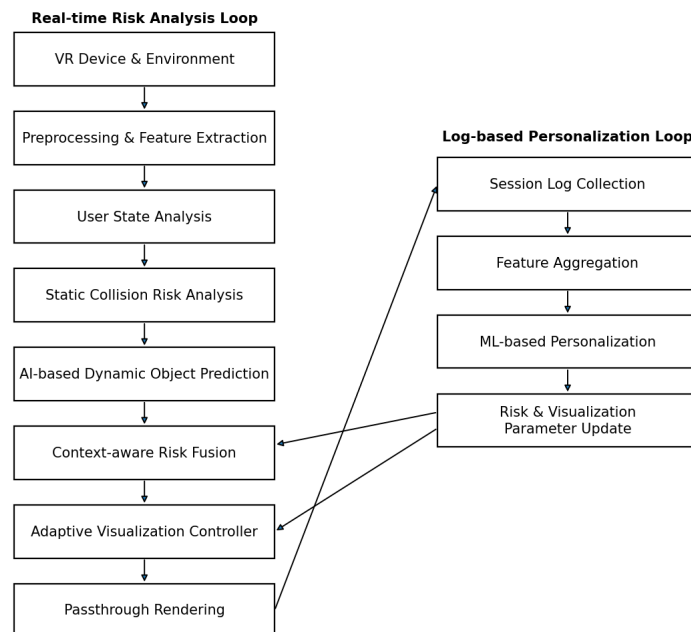


그림 3.1: 프로토타입 흐름도

본 시스템은 실시간 위험 인식, 상황별 시각화 제어, 로그 기반 사용자 적응 기능이 결합된 구조이다. 전체 시스템은 크게 실시간 위험 판단 루프와 세션 기반 개인화 루프로 구성된다.

실시간 위험 판단 루프에서는 HMD 및 컨트롤러 센서, Guardian 경계 정보, Passthrough Camera로부터 수집된 데이터를 바탕으로 사용자 상태와 주변 환경 정보를 분석한다. 먼저 사용자의 머리 및 컨트롤러 움직임을 기반으로 정적(Static), 동적(Dynamic), 격동적(Agitated) 상태를 분류하고, 시선 방향, 이동 속도, 조작 집중도 등을 함께 고려하여 사용자 상태 기반 위험도 R_{static} 를 산출한다.

정적 환경에 대해서는 Guardian 경계, 벽, 고정 장애물과의 거리, 접근 속도, 접근 방향, Time-To-Collision(TTC)을 이용하여 정적 충돌 위험도 $R_{collision}$ 를 계산한다. 또한 기존 Guardian 기반 시스템과 달리, 주변 사람, 반려동물, 이동 물체와 같은 동적 객체를 함께 고

려한다. 동적 객체는 Passthrough Camera 영상에서 객체 탐지 모델을 통해 인식하고, 객체 추적 과정을 통해 최근 움직임을 시계열 Feature로 구성한다. 이후 TCN-GRU 기반 AI 예측 모델을 활용하여 객체별 동적 충돌 위험도 $R_{dynamic}$ 과 접근 의도 기반 위험도 R_{intent} 를 산출한다.

이렇게 계산된 사용자 상태 위험도, 정적 및 동적 충돌 위험도, 동적 객체의 접근 의도 위험도는 Context-aware Risk Fusion 단계에서 통합되어 최종 위험도 R_{total} 로 산출된다. 시스템은 최종 위험도와 상황 맥락에 따라 단순 Passthrough On/Off뿐 아니라 위험 방향에 한정된 부분 Passthrough, Augmented Virtuality 오브젝트, 반투명 Volumetric Cue, Full Passthrough 등 다양한 시각화 방식을 선택적으로 적용한다. 이를 통해 사용자가 위험 상황을 인지할 수 있도록 하면서도 불필요한 현실 노출로 인한 몰입감 저하를 최소화한다.

세션 기반 개인화 루프에서는 Passthrough 작동 이력, 사용자의 수동 해제 이벤트, 시각화 지속 시간, 위험도 변화, 세션 정보 등을 로그로 저장한다. ML 기반 개인화 모델은 축적된 로그를 바탕으로 사용자별 행동 패턴과 선호도를 학습하고, 위험도 가중치와 시각화 전환 임계값을 점진적으로 조정한다. 이를 통해 동일한 위험 상황에서도 사용자 특성에 맞는 Adaptive Passthrough 제어가 가능하도록 한다.

3.2 사용 기술 및 전체 레이어 정의

본 시스템은 VR Client, AI/ML, Backend, Data, Dashboard의 5개 레이어로 구성된다. 각 레이어의 역할과 사용 기술은 다음과 같다.

3.2.1 VR Client Layer

사용자가 직접 경험하는 핵심 실행 환경으로, Unity와 C#, Meta XR SDK를 기반으로 구성된다. Unity는 Quest 3 앱 개발을 위한 메인 엔진으로 사용되며, C#은 센서 데이터 수집, 위험도 계산, Passthrough 제어 로직을 구현하는 언어이다. Meta XR SDK는 Guardian 경계, 컨트롤러 입력, HMD 추적을 담당하는 핵심 SDK이다. 이 레이어에서 수행되는 주요 처리는 IMU 센서 수집, 경계 거리 계산, 상태 분류 결과 반영, 실시간 Passthrough 시각화이다.

3.2.2 AI/ML Layer

AI/ML Layer는 상황 인식 기반 위험 분석과 로그 기반 사용자 개인화 제어를 담당한다. 본 레이어에서는 Passthrough Camera 영상으로부터 동적 객체를 인식하고, 객체의 이동 흐름을 분석하여 충돌 위험도와 접근 의도를 예측한다.

동적 객체 인식에는 YOLOv8n과 같은 경량 객체 탐지 모델을 사용하며, SORT 또는

ByteTrack 기반 추적 알고리즘을 통해 동일 객체의 이동 궤적을 추적한다. 추적된 객체의 최근 T 프레임 Feature는 TCN-GRU 기반 multi-task prediction model에 입력되며, 모델은 객체별 동적 위험도 $R_{dynamic}$ 과 접근 의도 클래스 $\hat{c}_{intent}$ 를 동시에 예측한다.

또한 사용자별 Passthrough 작동 이력, 수동 해제 이벤트, 세션 시간, 플레이 공간 크기 등의 로그를 분석하여 위험도 가중치와 시각화 전환 임계값을 조정한다. 이를 통해 동일한 위험 상황에서도 사용자 특성에 맞는 Adaptive Passthrough 제어가 가능해진다.

구현에는 PyTorch, scikit-learn, ONNX, Unity Sentis를 활용하며, 학습된 모델은 온디바이스 추론이 가능한 형태로 변환하여 VR Client Layer에 반영한다.

3.2.3 Backend Layer

VR 앱과 대시보드 사이의 데이터 허브 역할을 담당하며, FastAPI를 기반으로 REST API 중심으로 설계된다. 세션 생성, 설정값 조회, 로그 업로드, 결과 조회 API를 제공하며, Python 기반으로 구현되어 AI/ML 모델 관리 및 분석 스크립트와의 연결성이 높다.

3.2.4 Data Layer

실험 데이터와 설정값의 영구 저장소로, PostgreSQL을 사용한다. 저장되는 주요 데이터는 사용자별 세션 로그, 위험도 및 Passthrough 투명도 파라미터, 실험 조건이며, 향후 통계 분석 및 시각화에 활용된다.

3.2.5 Dashboard Layer

개발자의 운영 및 분석 인터페이스로, React를 기반으로 구성된다. 세션 타임라인 확인, 위험도 변화 시각화, 실험군 및 비교군 결과 조회 기능을 제공한다.

3.3 상황 인식 기반 위험 분석 및 시각화

본 시스템은 사용자의 안전 상태를 실시간으로 판단하기 위해 네 가지 위험도 항목을 산출하고 이를 통합한 최종 위험도 R_{total} 을 계산한다.

$$R_{total} = w_c R_{collision} + w_s R_{static} + w_d R_{dynamic} + w_i R_{intent} \quad (3.1)$$

여기서 R_{static} 은 사용자의 움직임 상태 기반 위험도, $R_{collision}$ 은 정적 경계와의 물리적 충돌 위험도, $R_{dynamic}$ 은 주변 동적 객체 기반 위험도, R_{intent} 은 동적 객체의 접근 의도 기반 위험도이다. 가중치 w_c, w_s, w_d, w_i 는 $\sum w = 1$ 을 만족하며, 초기값은 균등 설정 후 ML 개인화

모델(3.4 절)이 세션마다 사용자에게 맞게 갱신한다. 각 위험도의 구체적인 산출 방법은 이하 소절에서 순서대로 기술한다.

3.3.1 위험도 산출을 위한 기본 입력 Feature 정의

(1) 사용자 움직임 Feature

머리 선속도는 HMD 속도 벡터의 크기로, 사용자의 전반적인 이동 상태를 파악하는 데 사용된다.

$$v_h = \|\vec{v}_{head}\| \quad (3.2)$$

여기서 $\vec{v}_{head}$ 는 HMD의 속도 벡터이며, v_h 는 그 크기(노름)이다.

머리 가속도는 HMD 가속도 벡터의 크기로, 급격한 움직임을 탐지하는 데 활용된다.

$$a_h = \|\vec{a}_{head}\| \quad (3.3)$$

여기서 $\vec{a}_{head}$ 는 머리의 가속도 벡터이며, a_h 는 그 크기이다.

머리 각속도는 머리의 회전 속도 벡터의 크기로, 사용자가 얼마나 빠르게 시선을 회전하고 있는지를 나타낸다.

$$\omega_h = \|\vec{\omega}_{head}\| \quad (3.4)$$

여기서 $\vec{\omega}_{head}$ 는 머리의 회전 속도를 나타내는 벡터이며, ω_h 는 그 크기이다.

손 평균 속도는 양손 컨트롤러 속도의 평균으로, 사용자의 손 중심 활동 정도를 나타낸다.

$$v_c = \frac{\|\vec{v}_L\| + \|\vec{v}_R\|}{2} \quad (3.5)$$

여기서 $\vec{v}_L$, $\vec{v}_R$ 은 각각 왼손과 오른손 컨트롤러의 속도 벡터이며, v_c 는 양손의 평균 속도이다.

손-머리 움직임 비율은 손 움직임과 머리 움직임의 상대적 비율로, 값이 클수록 손 중심 활동이 크다는 의미이며 UI 조작이나 상호작용 상황을 구분하는 데 사용된다.

$$r_{hand/head} = \frac{v_c}{v_h + \epsilon} \quad (3.6)$$

여기서 v_c 는 손 평균 속도, v_h 는 머리 선속도이며, ϵ 은 분모가 0이 되는 것을 방지하기 위한 작은 값이다.

(2) 경계 기반 Feature

최소 경계 거리 d_{min} 은 사용자 위치와 가장 가까운 경계 사이의 거리로, 충돌 위험도를

판단하는 가장 기본적인 요소이다.

경계 방향 접근 속도는 사용자가 경계 방향으로 얼마나 빠르게 이동하고 있는지를 나타낸다. 음수 값은 경계에서 멀어지는 경우이므로 0으로 처리한다.

$$v_{toward} = \max(0, \vec{v} \cdot \hat{n}_b) \quad (3.7)$$

여기서 $\vec{v}$ 는 사용자의 속도 벡터이며, $\hat{n}_b$ 는 사용자에서 경계를 향하는 단위 방향 벡터이다. 두 벡터의 내적을 통해 경계 방향 성분만을 추출한다.

경계 방향 접근 가속도는 사용자가 경계 방향으로 얼마나 빠르게 속도를 증가시키고 있는지를 나타내며, 단순 속도뿐 아니라 급격한 움직임까지 반영한다.

$$a_{toward} = \max(0, \vec{a} \cdot \hat{n}_b) \quad (3.8)$$

여기서 $\vec{a}$ 는 사용자의 가속도 벡터이며, $\hat{n}_b$ 는 앞서와 동일한 경계 방향 단위 벡터이다.

(3) 시선 기반 Feature

시선 방향과 경계 방향 사이의 각도를 계산하여 사용자가 해당 경계를 인지하고 있는지를 여부를 판단한다. 각도가 클수록 사용자가 인지하지 못하는 사각지대에 해당한다.

$$\theta = \cos^{-1} \left(\frac{\vec{f} \cdot \vec{b}}{\|\vec{f}\| \|\vec{b}\|} \right) \quad (3.9)$$

여기서 $\vec{f}$ 는 사용자의 시선 방향 벡터이고, $\vec{b}$ 는 경계 방향 벡터이다.

3.3.2 충돌 위험도($R_{collision}$)

Potential Field 기반 거리 모델은 장애물과의 거리 감소에 따라 위험도가 비선형적으로 증가하는 모델이다. 다만 원래 퍼텐셜 필드는 로봇의 물리적 이동을 위한 힘 생성 모델이기 때문에 VR 환경에서는 직접 적용이 어렵다. 이에 해당 모델의 핵심 개념인 “거리 감소에 따른 위험도 증가” 특성만 유지하고, 위험도를 0~1 범위로 정규화한 선형 모델로 단순화하여 적용했다.

$$R_d = 1 - \min \left(1, \frac{d_{min}}{d_{safe}} \right) \quad (3.10)$$

여기서 d_{min} 은 현재 사용자와 경계 사이의 최소 거리이고, d_{safe} 는 시스템이 정한 안전 거리이다. d_{min}/d_{safe} 는 현재 거리가 안전 거리의 몇 배인지를 나타내며, 이 값이 1 이상이면 안전 거리 밖에 있다는 뜻이다. $\min(1, \cdot)$ 을 사용해 최댓값을 1로 제한하고, $1 -$ 을 취하는 이유는 거리가 가까울수록 위험도가 커지도록 방향을 뒤집기 위해서이다. 예를 들어 $d_{min} = d_{safe}$

이면 $R_d = 0$ 으로 안전 거리 밖이므로 위험도는 0이며, $d_{min} = 0$ 이면 $R_d = 1$ 로 경계에 거의 닿은 상태이므로 위험도는 1이다.

Time-To-Collision (TTC)은 충돌까지 남은 시간을 나타내는 물리적 지표로, 단순 거리 기반 판단의 한계를 보완한다. 동일 거리에서도 접근 속도에 따라 위험도를 다르게 평가할 수 있다.

$$TTC = \frac{d_{min}}{v_{toward} + \epsilon} \quad (3.11)$$

여기서 d_{min} 은 사용자와 가장 가까운 경계 사이의 거리이며, v_{toward} 는 경계 방향으로의 접근 속도이다. 음수 값은 충돌 위험이 없으므로 0으로 처리하며, ϵ 은 분모가 0이 되는 것을 방지하기 위한 작은 값이다. 예를 들어 사용자가 경계에서 0.6m 떨어져 있고 경계 방향으로 0.3m/s의 속도로 이동하고 있다면 $TTC = 0.6/0.3 = 2$, 즉 현재 속도가 유지되면 약 2초 후 경계에 도달한다는 의미이다.

TTC는 시간 단위의 물리적 값이므로 직접 위험도로 사용하기 어렵다. 따라서 기준 시간 T_{safe} 로 정규화하고, 값이 작을수록 위험도가 커지도록 다음과 같이 변환한다.

$$R_{TTC} = 1 - \min\left(1, \frac{TTC}{T_{safe}}\right) \quad (3.12)$$

여기서 T_{safe} 는 안전하다고 판단하는 기준 시간이다.

R_a 는 가속도 기반 위험도로, 경계 방향으로의 가속도를 기준으로 한다. 값이 클수록 급격한 움직임으로 판단하여 위험도를 높인다.

$$R_a = \min\left(1, \frac{a_{toward}}{a_{max}}\right) \quad (3.13)$$

여기서 a_{toward} 는 경계 방향 접근 가속도이며, a_{max} 는 시스템이 정한 최대 기준 가속도이다.

R_{blind} 는 사각지대 위험도로, 시선 방향과 경계 방향의 각도 θ 를 기반으로 위험도를 부여한다. 사용자가 보지 못하는 방향일수록 높은 위험도를 부여한다.

$$R_{blind} = \begin{cases} 0.2 & \theta < 60^\circ \quad (\text{정면}) \\ 0.5 & 60^\circ \leq \theta < 120^\circ \quad (\text{측면}) \\ 0.8 & \theta \geq 120^\circ \quad (\text{후방}) \end{cases} \quad (3.14)$$

여기서 θ 는 앞서 정의한 시선-경계 간 각도이다.

다중 요인 가중합 모델은 여러 위험 요소를 결합하기 위해 각 요소에 가중치를 부여하여

합산하는 방식이다.

$$R = \sum_i w_i x_i \quad (3.15)$$

여기서 w_i 는 각 요소의 가중치이며 x_i 는 각 위험 요소의 정규화된 값이다.

이에 따라 위의 이론적 기반을 종합하여 충돌 위험도를 다음과 같이 정의한다.

$$R_{collision} = w_d R_d + w_{ttc} R_{TTC} + w_a R_a + w_{blind} R_{blind} \quad (3.16)$$

여기서 $w_d, w_{ttc}, w_a, w_{blind}$ 는 각 요소의 가중치이며, $\sum w_i = 1$ 을 만족하도록 설정한다. 각 위험도 항목은 다음과 같다. R_d 는 경계까지의 거리 기반 위험도, R_{TTC} 는 현재 속도로 경계에 충돌하기까지 남은 시간 기반 위험도, R_a 는 경계 방향 접근 가속도 기반 위험도, R_{blind} 는 사용자의 시선과 경계 방향 간 각도에 따른 사각지대 위험도이다.

3.3.3 정적 경계 기반 위험도 (R_{static})

본 시스템은 사용자의 움직임 패턴을 **정적(Static)**, **동적(Dynamic)**, **격동적(Agitated)**의 세 가지 상태로 분류하여 위험도 산출 및 Passthrough 제어에 반영한다. 각 상태는 머리 선속도 v_h , 머리 가속도 a_h , 이동 지속 시간을 기준으로 판별하며, 분류 결과는 최종 위험도 수식의 R_{static} 항에 반영된다. 상태 판별은 단일 프레임이 아닌 최근 n 프레임의 평균 속도와 최대 속도를 함께 활용하여 일시적인 노이즈에 의한 오분류를 방지한다. 정적 상태는 사용자가 제자리에 있는 구간으로, Passthrough를 억제하여 몰입감을 유지한다. 동적 상태는 걷기, 고개 돌리기 등 일반적인 활동에 해당하며, 충돌 위험도를 종합하여 활성화 여부를 결정한다. 격동적 상태는 순간적으로 빠른 이동이나 급격한 회전이 감지된 경우로, Passthrough를 즉시 활성화한다.

표 3.1: 사용자 상태 분류 기준 및 Passthrough 제어 방향

상태	분류 기준	R_{static}	Passthrough 제어 방향
정적 (Static)	$v_h < 0.05$ m/s $a_h \approx 0$ 이동 지속 시간 없음	0.0	Passthrough 억제. 단, $d_{min} \leq d_{safe}$ 이면 예외 적용
동적 (Dynamic)	$0.05 \leq v_h \leq 1.0$ m/s 이동 지속 시간 > 0.5 s	0.5	$R_{collision}$ 점수에 따라 조건부 활성화
격동적 (Agitated)	$v_h > 1.0$ m/s, 또는 $a_h > 5.0$ m/s ²	1.0	Passthrough 즉시 활성화. R_a, R_{TTC} 가중치 상향 적용

3.3.4 AI 기반 동적 객체 충돌 및 접근 의도 예측 기반 위험도 ($R_{dynamic}$, R_{intent})

본 시스템은 기존 Guardian 기반 시스템과 달리 정적 경계뿐 아니라 주변 사람, 반려동물, 이동 물체와 같은 동적 객체를 함께 고려한다.

동적 객체는 시간에 따라 위치와 속도가 변하므로, 단순 거리 기반 판단만으로는 충분하지 않다. 따라서 본 시스템은 객체 탐지, 객체 추적, AI 기반 예측을 결합하여 동적 객체 위험도를 산출한다.

동적 객체는 Passthrough Camera API로부터 획득한 영상에서 YOLOv8n과 같은 경량 객체 탐지 모델을 통해 인식한다. 이후 SORT 또는 ByteTrack 기반 추적 알고리즘을 사용하여 프레임 간 동일 객체에 ID를 부여하고, 최근 T 프레임 동안의 움직임을 시계열 Feature로 구성한다.

표 3.2: 동적 객체 예측 Feature

Feature	Symbol	설명
Relative Distance	d_i	사용자와 객체 i 사이의 거리
Approach Velocity	$v_{close,i}$	객체 i 가 사용자 방향으로 가까워지는 속도
Dynamic TTC	TTC_i^{dyn}	현재 이동이 유지될 경우 예상 충돌까지 남은 시간
Approach Angle	θ_i	사용자 기준 객체 i 의 접근 각도
Bounding Box Size	s_i	카메라 영상 내 객체 i 의 Bounding Box 크기
Bounding Box Change	Δs_i	Bounding Box 크기 변화량으로, 객체의 접근 여부 추정에 활용
Object Type	c_i	사람, 반려동물, 공, 이동 물체 등 객체 유형
User Velocity	v_{user}	사용자의 이동 속도
User Head Rotation	ω_{user}	사용자의 머리 회전 속도 및 주의 방향 추정에 활용

객체 i 에 대한 최근 T 프레임 동안의 시계열 입력 Feature는 표 3.2에 정의한 값들을 이용하여 다음과 같이 구성한다.

$$\mathbf{x}_i^{t-T:t} = [d_i, v_{close,i}, TTC_i^{dyn}, \theta_i, s_i, \Delta s_i, c_i, v_{user}, \omega_{user}]^{t-T:t} \quad (3.17)$$

객체 유형 c_i 는 모델 입력을 위해 One-hot Encoding 또는 Embedding 형태로 변환하여 사용한다.

본 시스템은 TCN-GRU 기반 Multi-task Prediction Model을 사용한다. TCN은 짧은 시간 구간의 움직임 패턴을 추출하고, GRU는 시간에 따른 접근 흐름을 요약한다. 모델은 객체별 동적 충돌 위험도와 접근 의도 클래스를 동시에 예측한다.

$$\mathbf{h}_i = \text{GRU}(\text{TCN}(\mathbf{x}_i^{t-T:t})) \quad (3.18)$$

$$\hat{R}_{dynamicAI,i} = \sigma(\mathbf{W}_r \mathbf{h}_i + \mathbf{b}_r) \quad (3.19)$$

$$\hat{\mathbf{y}}_{intent,i} = \text{softmax}(\mathbf{W}_c \mathbf{h}_i + \mathbf{b}_c) \quad (3.20)$$

여기서 $\hat{R}_{dynamicAI,i}$ 는 AI 모델이 예측한 동적 충돌 위험도이며, $\hat{\mathbf{y}}_{intent,i}$ 는 객체의 접근 의도 분류 결과이다.

AI 예측의 안정성을 높이기 위해 거리, 접근 속도, TTC 기반의 규칙 기반 위험도를 함께 사용한다. 여기서 $R_{d,i}^{dyn}$, $R_{v,i}^{dyn}$, $R_{TTC,i}^{dyn}$ 는 각각 객체 i 의 상대 거리, 사용자 방향 접근 속도, 동적 TTC를 0에서 1 사이로 정규화한 위험도이다.

$$R_{dynamicRule,i} = w_d R_{d,i}^{dyn} + w_v R_{v,i}^{dyn} + w_t R_{TTC,i}^{dyn} \quad (3.21)$$

최종 객체별 동적 충돌 위험도는 AI 예측값과 규칙 기반 위험도를 결합하여 계산한다.

$$R_{dynamic,i} = \lambda \hat{R}_{dynamicAI,i} + (1 - \lambda) R_{dynamicRule,i} \quad (3.22)$$

여러 동적 객체가 동시에 감지되는 경우, 가장 높은 위험도를 가지는 객체를 전체 동적 충돌 위험도로 사용한다.

$$R_{dynamic} = \max_i R_{dynamic,i} \quad (3.23)$$

동적 객체에 대한 판단은 단순히 충돌 가능성만으로 충분하지 않다. 예를 들어 사람이 사용자 주변을 단순히 지나가는 경우와, 사용자를 인식하고 대화나 도움 요청을 위해 접근하는 경우는 물리적 충돌 위험도는 비슷하게 나타날 수 있지만, 사용자에게 제공해야 하는 현실 인식 수준은 서로 다르다. 따라서 본 시스템은 동적 객체의 물리적 충돌 가능성을 나타내는 $R_{dynamic}$ 과 별도로, 객체의 접근 맥락과 상호작용 가능성을 나타내는 R_{intent} 를 함께 산출한다.

특히 사람 객체의 경우, 이동 방향, 사용자와의 거리 변화, 접근 지속 시간, 사용자 기준 접근 각도, 속도 변화 등을 종합하여 단순 통과, 주변 배회, 사용자 접근, 상호작용 의도, 충돌 위험 상태로 분류한다. 이 분류 결과는 모든 주변 객체를 항상 시각화하지 않고, 사용자가 실제로 인지해야 할 가능성이 높은 객체만 선택적으로 시각화하기 위한 기준으로 사용된다.

표 3.3: 동적 객체 접근 의도 분류

Intent Type	설명	위험도
Passing-by	사용자 주변을 단순히 지나감	0.1
Wandering	주변에 머무르지만 접근하지 않음	0.3
Approaching User	사용자 방향으로 접근함	0.6
Interaction-seeking	상호작용 의도를 가지고 접근함	0.8
Collision Threat	충돌 가능성이 높은 접근 상태	1.0

AI 모델의 의도 분류 결과는 가장 높은 확률을 가지는 클래스로 결정한다.

$$\hat{y}_{intent,i} = \arg \max_k \hat{y}_{intent,i,k} \quad (3.24)$$

의도 클래스는 위험도 점수 함수 $S(\cdot)$ 를 통해 수치화되며, 여러 객체가 동시에 존재하는 경우 가장 높은 의도 위험도를 사용한다.

$$R_{intent} = \max_i S(\hat{y}_{intent,i}) \quad (3.25)$$

3.3.5 다양한 Passthrough 시각화 방식

본 프로젝트는 단순히 다양한 Passthrough 시각화 방식을 구현하는 것에 그치지 않고, 각 시각화 방식이 사용자 경험에 미치는 영향을 체계적으로 비교 및 평가하는 것을 목표로 한다.

특히 위험 상황에서 현실 정보를 어느 수준까지 제공해야 안전성과 몰입감 사이의 균형을 효과적으로 유지할 수 있는지 분석하기 위해, 다양한 시각화 방식을 비교 실험 형태로 구성할 예정이다.

비교 대상은 다음과 같다.

- 위험 방향에 한정된 부분 Passthrough(Directional Passthrough)
- 현실 객체를 가상 환경 내부 오브젝트로 치환하는 Augmented Virtuality 방식
- 반투명 볼류메트릭 표현을 통한 실세계 객체 존재 암시 방식

각 방식은 충돌 상황 인지 속도, 몰입감(Presence), 현실 객체 인지 정확도, 사용자 선호도, 인지 부담(Cognitive Load) 등의 지표를 기준으로 비교 평가할 예정이다. 이를 통해 단순한 안전 기능 구현을 넘어, XR 환경에서의 현실 인식 방식이 사용자 경험에 미치는 영향을 분석하고자 한다.

이러한 비교 결과를 기반으로, 상황별 위험 수준과 사용자 상태에 따라 가장 적절한 시각화 방식을 선택적으로 적용하는 Adaptive Visualization 전략을 설계한다.

표 3.4: 시각화 방식 비교 평가 항목

시각화 방식	몰입감	안전성	인지 정확도
Directional Passthrough	높음	중간 이상	중간
Augmented Virtuality	높음	중간	높음
Volumetric Cue	매우 높음	중간	낮음

3.4 ML 기반 개인화 Passthrough 제어

이는 사용자마다 다른 환경과 행동 패턴을 학습하여 Passthrough 활성화 기준을 점진적으로 개인화하는 것이다. 상황 인식 기반 위험 분석 및 시각화 접근(3.3절)이 여러 센서 데이터를 바탕으로 즉각적인 위험 판단을 수행한다면, ML 개인화 제어는 세션이 누적될수록 해당 사용자에게 최적화된 Passthrough 환경을 조성하는 것을 목표로 한다.

Meta Quest 환경에서는 OpenXR API 및 Unity XR Toolkit을 통해 Passthrough 상태 이력, 컨트롤러 입력값, HMD 위치, 가속도 등의 데이터에 접근할 수 있으며, 수집된 로그는 로컬 JSON 파일로 저장하거나 REST API를 통해 외부 서버로 전송하여 관리할 예정이다.

3.4.1 로그 수집 구조

수집 로그는 행동 로그와 세션 로그 두 가지로 구분되며, 이벤트 발생 시점마다 타임스탬프와 세션 ID를 키로 묶어 JSON 형태로 직렬화한다. 각 항목의 정의 및 해석 기준은 표 3.5와 같다.

표 3.5: 로그 항목 및 해석 기준

분류	데이터 항목	기호	해석 기준
행동 로그	Passthrough 수동 해제 횟수	N_{cancel}	불필요한 활성화로 판단 → 임계값 상향
	Passthrough 활성화 횟수	$N_{activate}$	윈도우 내 활성화 빈도 산출에 사용
	Passthrough 지속 시간	$\bar{t}_{pt}$	2초 미만 시 불필요한 활성화로 분류
	컨트롤러 유희 시간	—	3초 이상 유희 시 현실 확인 필요 구간으로 해석
	평균/최대 이동 속도	$\bar{v}_h, v_{h,max}$	세션 중 IMU 센서값을 윈도우 단위로 집계
세션 로그	플레이 공간 크기	A_{space}	협소할수록 충돌 위험 증가 → Passthrough 민감도 가중치 반영
	누적 세션 시간	$T_{session}$	길수록 피로도 누적으로 판단, Min-Max 정규화 적용

수집된 로그는 슬라이딩 윈도우(Window Size W , Stride S) 단위로 집계되어 모델 입력

Feature 벡터 $\mathbf{x}$ 로 변환된다. 각 항목의 정의는 다음과 같다.

$$\mathbf{x} = [f_{pt}, r_{cancel}, \bar{t}_{pt}, \bar{v}_h, v_{h,max}, \tilde{A}_{space}, \tilde{T}_{session}]^T \in \mathbb{R}^7 \quad (3.26)$$

여기서 $f_{pt} = N_{activate}/W$ 는 윈도우 내 Passthrough 활성화 빈도, $r_{cancel} = N_{cancel}/(N_{activate} + \epsilon)$ 은 수동 해제 비율, $\bar{t}_{pt}$ 는 평균 Passthrough 지속 시간, $\bar{v}_h$ 와 $v_{h,max}$ 는 세션 중 IMU 센서값을 윈도우 단위로 집계한 평균 및 최대 이동 속도, $\tilde{A}_{space}$ 와 $\tilde{T}_{session}$ 은 각각 플레이 공간 크기와 누적 세션 시간의 Min-Max 정규화값이다.

3.4.2 개인화 모델 구조

초기 세션(데이터 부족 구간)에는 Cold-start 문제를 완화하기 위해 모든 사용자에게 동일한 기본 가중치 $\mathbf{w}_{Default}$ 를 적용하며, 누적 세션 수가 기준값 N_{min} 을 초과한 시점부터 개인화 모델의 출력으로 전환한다.

모델은 경량 지도 학습 분류기를 기반으로 하며, 구현 단계에서 다음 두 가지 후보를 비교하여 최종 채택한다.

표 3.6: 개인화 모델 후보 비교

모델	장점	단점
Random Forest (scikit-learn)	소량 데이터에서도 안정적, Feature 중요도 해석 가능, 온디바이스 추론 경량	시계열 패턴 반영 어려움
1D-CNN + FC Layer (PyTorch)	시계열 내 국소 패턴 학습 가능, 세션 간 변화 포착	데이터 충분히 쌓여야 성능 발휘

초기 프로토타입에서는 데이터 효율성을 고려하여 Random Forest를 우선 적용하고, 세션 로그가 충분히 누적된 이후 1D-CNN으로의 전환을 검토한다.

3.4.3 학습 레이블 생성

별도의 명시적 피드백 없이 사용자의 암묵적 행동으로부터 학습 레이블을 자동 생성한다. Passthrough가 활성화된 구간에 대해 다음 규칙을 적용하여 필요한 *활성화(Positive)* 또는 불필요한 *활성화(Negative)*로 분류한다.

- **Negative:** 활성화 후 2초 이내에 수동으로 해제된 경우, 활성화 시점의 컨트롤러 유휴 시간이 3초 미만인 경우 등
- **Positive:** 활성화 지속 시간이 3초 이상이며 수동 해제가 없는 경우, R_{total} 이 임계값을 초과한 직후 활성화된 경우 등

이렇게 생성된 레이블을 기반으로 모델은 어떤 상황에서 Passthrough 활성화가 실제로 필요했는지를 학습하며, 학습 결과는 다음 세션의 τ_{viz} 조정에 반영된다.

3.4.4 온디바이스 배포 및 반영

학습이 완료된 모델은 다음 파이프라인을 통해 Quest 3 온디바이스에 배포된다.

1. **학습**: 외부 서버(Backend Layer)에서 scikit-learn 또는 PyTorch로 학습
2. **변환**: ONNX 포맷으로 변환하여 모델 크기 최소화 (목표: 50 MB 이하)
3. **배포**: Unity의 Sentis(구 Barracuda) 런타임을 통해 온디바이스 추론
4. **반영**: 추론 결과인 $\mathbf{w}_{personal}$ 과 τ_{viz} 를 매 세션 시작 시 위험도 산출 모듈에 로드

추론은 매 프레임이 아닌 슬라이딩 윈도우 단위(예: 5초마다)로 수행하여 온디바이스 연산 부하를 최소화한다. 이 과정이 세션마다 반복되면서 모델은 해당 사용자의 패턴에 점진적으로 수렴한다.

제4장 비교 평가 계획

본 프로젝트는 단순 충돌 방지 성능뿐 아니라, 몰입감과 현실 인지 사이의 균형을 함께 평가하는 것을 목표로 한다. 이를 위해 기존 Guardian 기반 시스템과 Full Passthrough 방식, 그리고 본 연구에서 제안하는 Adaptive Passthrough Framework를 비교 대상으로 설정한다. 비교 평가는 동일한 XR 환경에서 수행되며, 사용자의 움직임 및 위험 상황에 대한 반응 특성을 기반으로 분석한다.

표 4.1: 비교 평가 대상

비교 대상	설명
Baseline Guardian	기존 경계 기반 Passthrough 시스템
Full Passthrough	전체 현실 환경 상시 노출 방식
Proposed Framework	제안 Adaptive Passthrough 시스템

평가 지표는 안전성, 몰입감, 사용자 경험 측면을 종합적으로 반영하도록 구성한다. Collision Rate와 Reaction Time은 위험 상황 대응 성능을 측정하기 위한 정량 지표이며, Presence Score와 User Preference는 사용자의 주관적 경험을 평가하기 위한 지표이다. 또한 Cognitive Load를 함께 측정하여, 현실 정보 제공 방식이 사용자 인지 부담에 미치는 영향을 분석한다.

표 4.2: 평가 항목

평가 지표	설명
Collision Rate	충돌 발생 빈도
Reaction Time	위험 상황 반응 시간
Presence Score	몰입감 수준
User Preference	사용자 선호도
Cognitive Load	인지 부담 수준

실험 결과를 바탕으로, 제안 시스템이 기존 Guardian 기반 방식 대비 충돌 위험 감소와 현실 인지 향상에 어떠한 영향을 미치는지 분석하고, 동시에 몰입감 유지 측면에서의 장단점을 함께 평가한다.

제5장 기타

5.1 개발 일정 및 역할 분배

본 프로젝트는 2025년 5월부터 9월 18일까지 약 5개월간 진행되며, 설계, 구현, 검증의 세 단계로 나누어 수행한다.

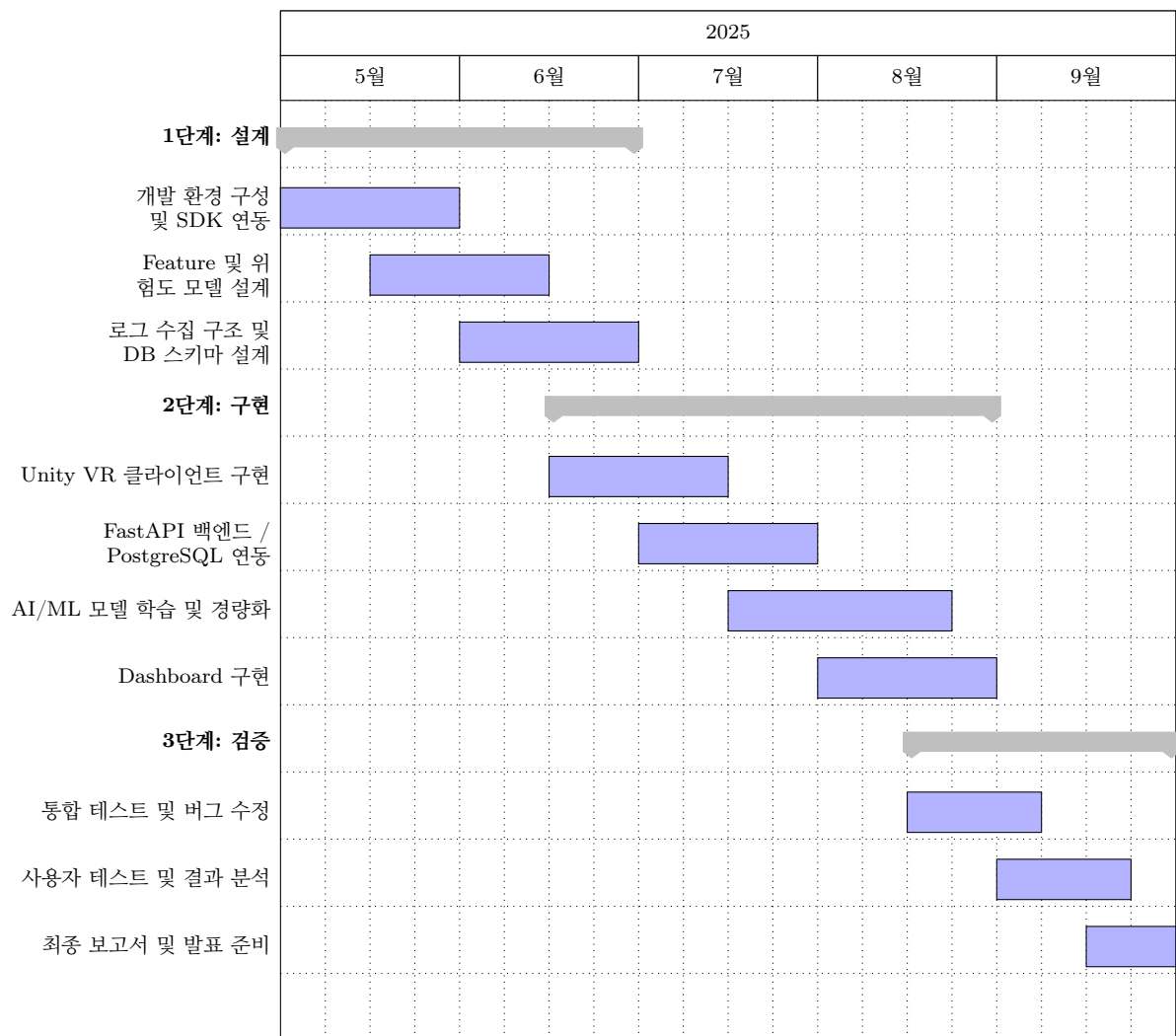


그림 5.1: 프로젝트 개발 일정

팀원	담당 역할	세부 내용
따다소 (팀장)	보고서 작성 및 개인화 제어 연구	● 로그 수집 및 피드백 전략 수집 ● ML 기반 개인화 모델 구현 및 경량화 ● 사용자 테스트 진행 및 결과 분석
최아영	센서 데이터 수집 및 Feature 알고리즘 개발	● 센서 데이터 수집 방법 및 전처리 ● Feature 및 위험도 계산 알고리즘 구현 ● 위험도 산출 로직 모델 설계 및 구현
이승주	전체 시스템 레이어 구조 구축 및 흐름 설계	● 전체 레이어 구조 및 흐름 설계 ● Fast API 백엔드/PostgreSQL 연동 ● Dashboard 및 테스트용 시스템 구현

5.2 기대 효과

본 프로젝트에서 제안하는 Passthrough 제어 시스템은 상황 인식 기반 위험 분석 접근과 로그 피드백 기반 ML 최적화를 결합함으로써, VR 환경에서의 안전성과 몰입도를 동시에 향상시킬 수 있다. 사전에 정의된 규칙을 통해 충돌 위험에 즉각적으로 대응하면서도, 사용자 행동 로그와 환경 데이터를 기반으로 개인별 Passthrough 활성화 패턴을 학습하여 불필요한 개입은 줄이고 필요한 순간에는 보다 정확하게 반응할 수 있다. 또한 암묵적 피드백과 비침습적 인터랙션을 활용함으로써 사용자 경험을 방해하지 않으면서 지속적인 성능 개선이 가능하며, 결과적으로 사용자 맞춤형 XR 인터페이스 구현과 안전한 몰입 환경 조성에 기여할 것으로 기대된다. 또한 위험 상황별로 서로 다른 시각화 방식을 선택적으로 적용함으로써, 불필요한 현실 노출을 줄이면서도 필요한 시점의 안전 확보를 강화할 수 있다.

참고문헌

- [1] 김성진. 확장현실(xr) 산업의 현황과 과제. Kiet 산업경제, 산업연구원(KIET), 2023.
- [2] Meta. Meta quest에서 패스스루를 사용하는 방법, n.d.
- [3] Meta for Developers. Passthrough camera API overview. Meta Horizon Documentation, <https://developers.meta.com/horizon/documentation/spatial-sdk/spatial-sdk-pca-overview/>, December 2025.
- [4] ArborXR. How to disable the boundary on meta quest devices. <https://help.arborxr.com/en/articles/8638529-how-to-disable-the-boundary-on-meta-quest-devices>, 2024.
- [5] Meta Platforms, Inc. Meta horizon platform v66 reference, 2024. Accessed: 2026-04-25.
- [6] Arkio. Working in mixed reality. <https://support.arkio.is/hc/en-us/articles/7187427474205-Working-in-Mixed-Reality>, 2026.